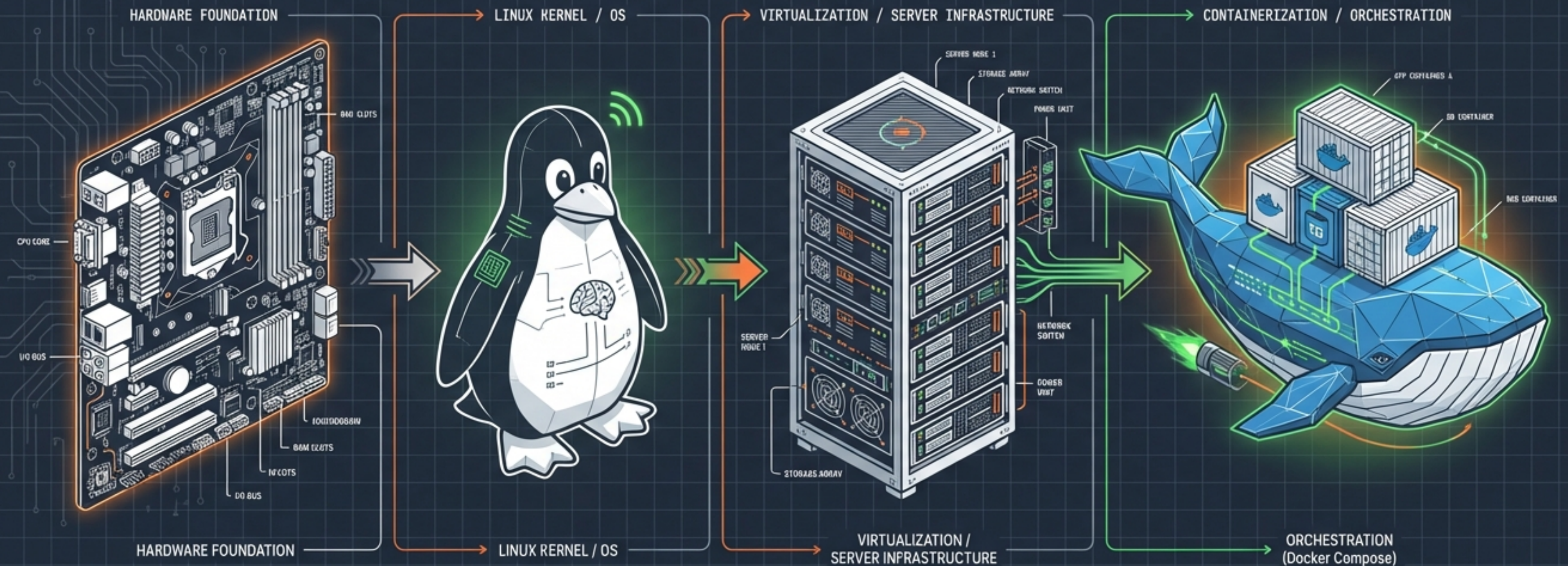
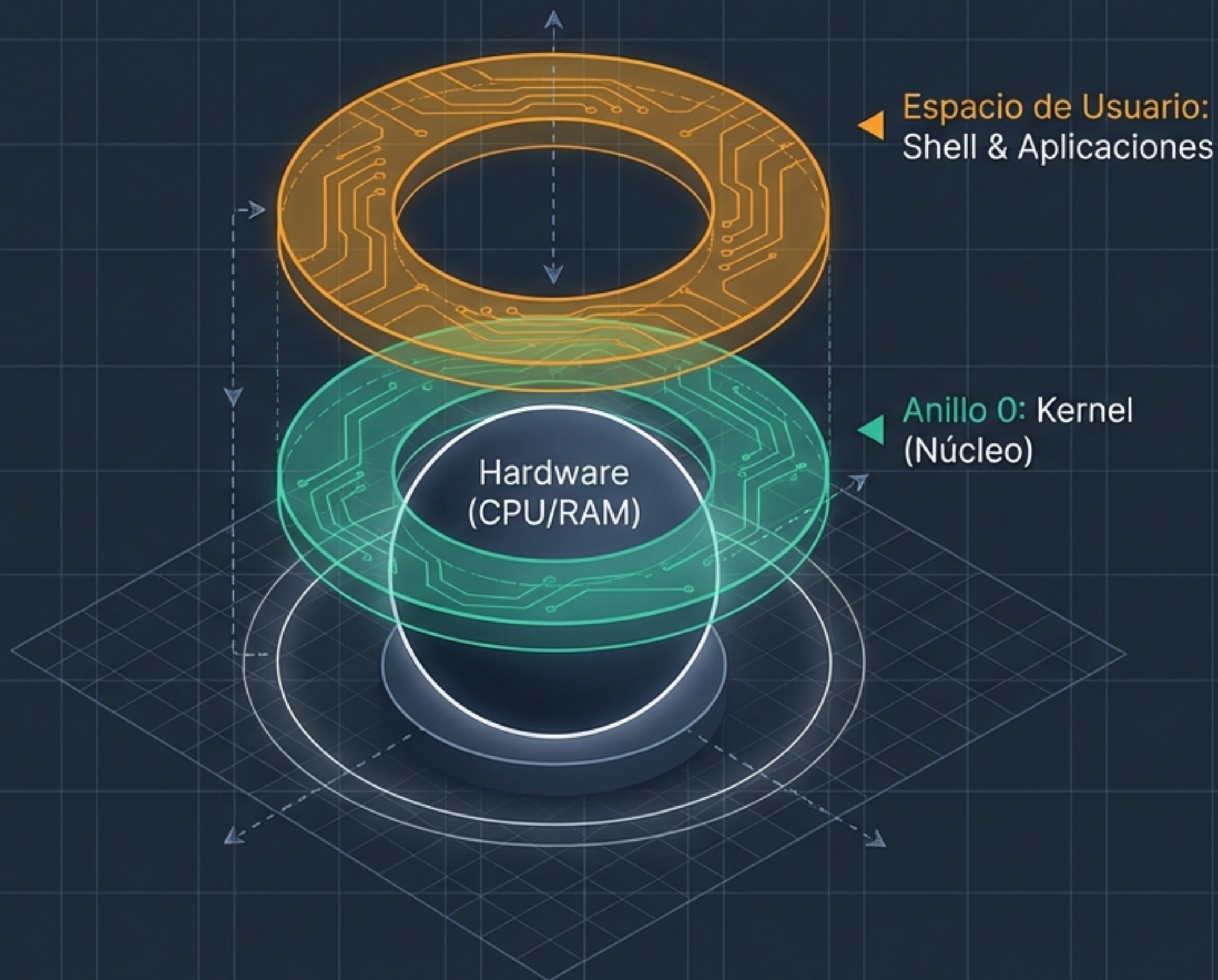


De la Máquina al Contenedor: Fundamentos de Despliegue

Una guía visual desde el Kernel de Linux hasta la orquestación con Docker Compose.





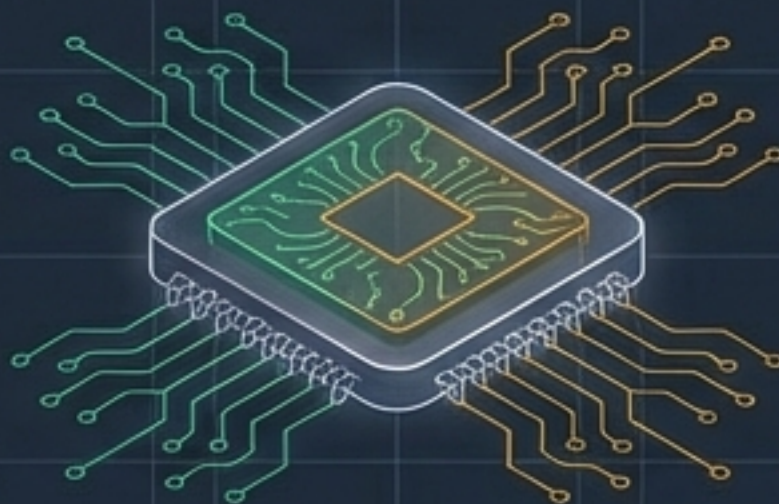
El Director de la Orquesta: El Sistema Operativo

- **Kernel (Núcleo):** El corazón del sistema. Gestiona memoria y hardware directamente.
- **Shell:** El intérprete (CLI/GUI). Traduce tus órdenes para el Kernel.
- **Driver:** El traductor entre el OS y los periféricos.

GNU/Linux y sus Distribuciones



GNU



Linux Kernel



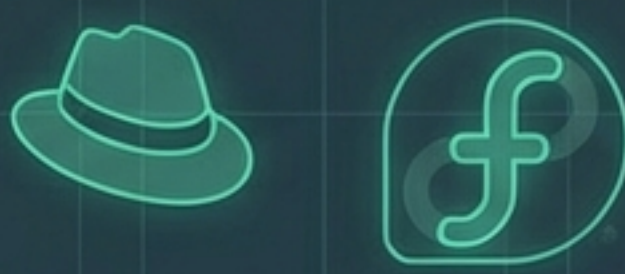
Sistema Operativo

Debian / Ubuntu



Estándar de desarrollo.
Paquetes **.deb**. Amigable.

Red Hat / Fedora



Enfoque empresarial.
Estabilidad.

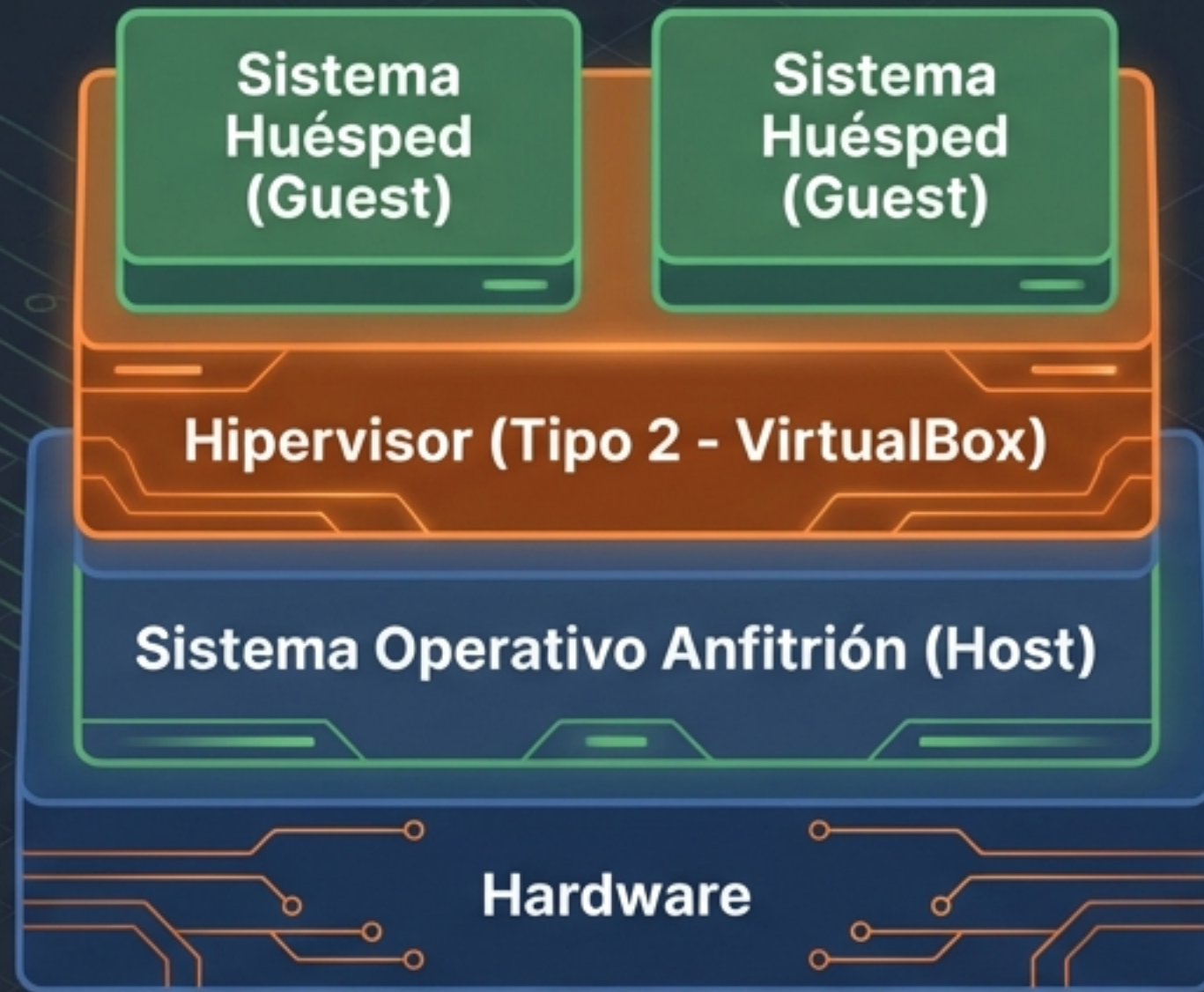
Alpine Linux



Crucial para **Docker**.
Minimalista, ligera y segura.

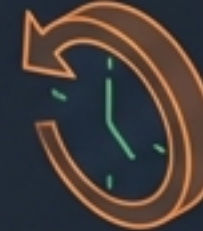
Una distribución = Kernel Linux + Utilidades preconfiguradas.

Virtualización: La Máquina dentro de la Máquina



Aislamiento:

Fallos contenidos.



Snapshots:

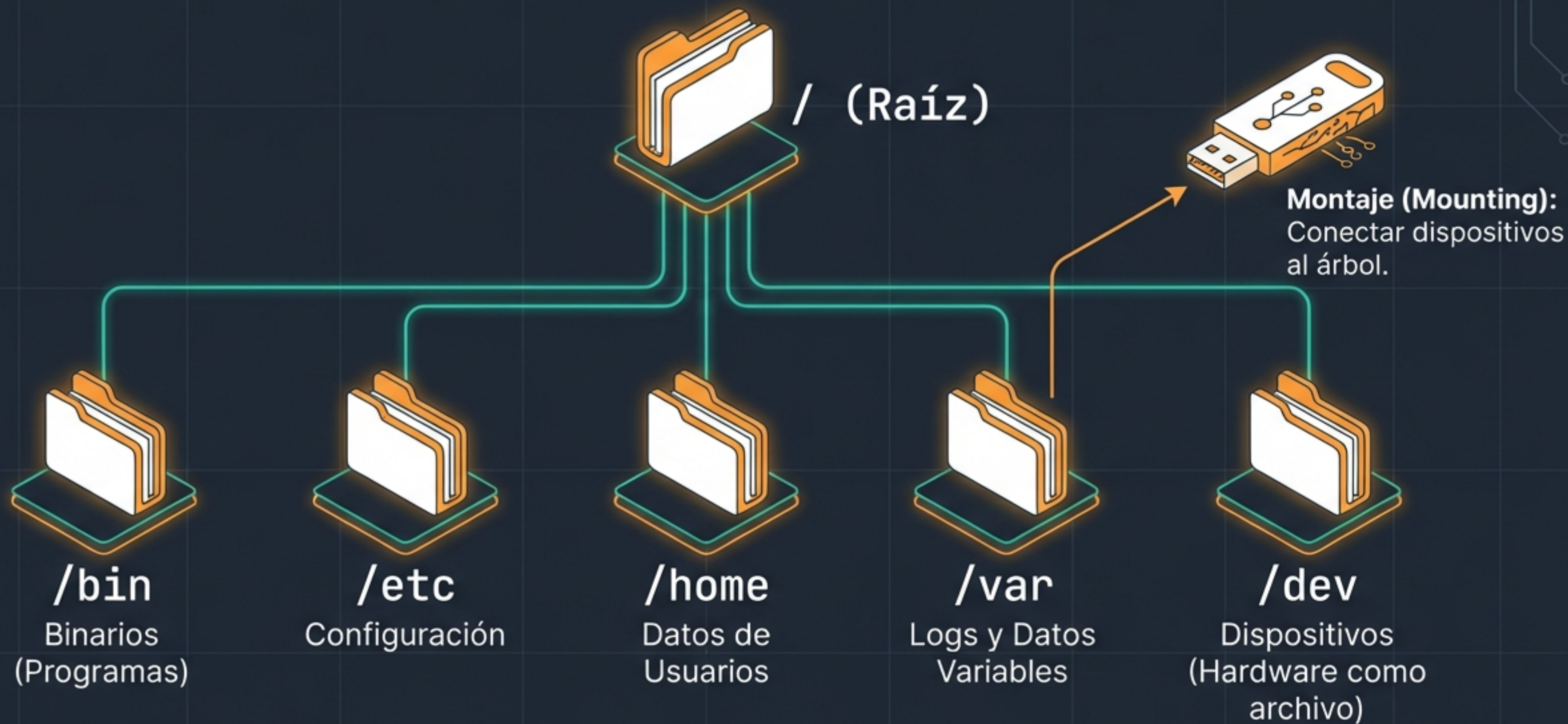
Viajes en el tiempo.



Sandboxing:

Pruebas seguras.

La Jerarquía: En Linux, Todo es un Archivo



Guardianes del Sistema: Usuarios y Permisos

r = Read (Leer)

w = Write
(Escribir)

x = Execute
(Ejecutar)

-rwxr-xr--

Propietario
(Owner)

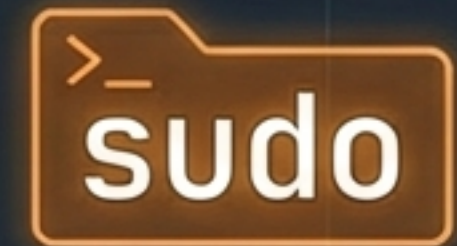
Grupo

Otros



ROOT

Superusuario (Anillo 0)
- Poder total.



Gestión de Software: Paquetes y Procesos

PAQUETES (APT)



PROCESOS

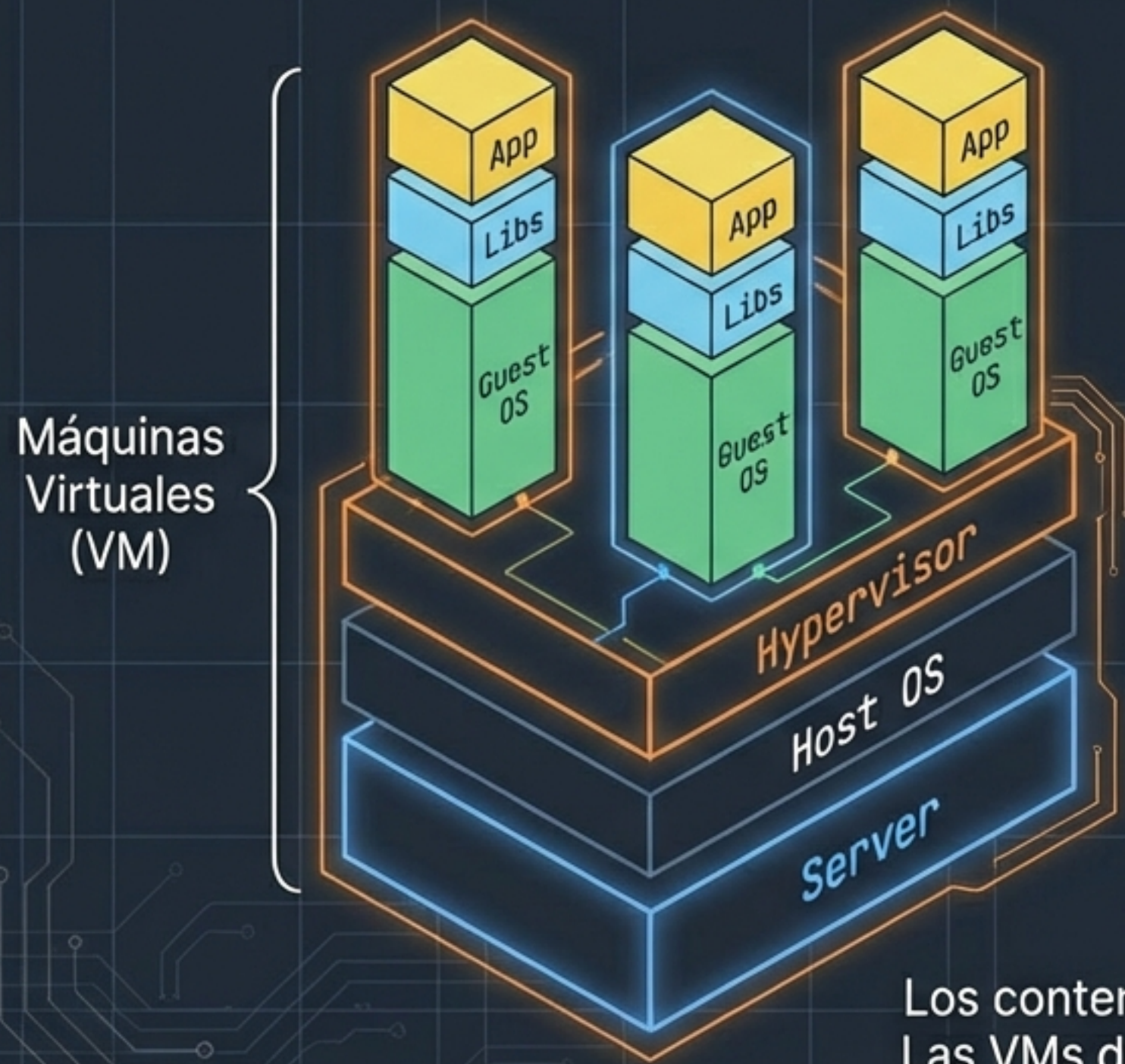


PID:
ID de Proceso

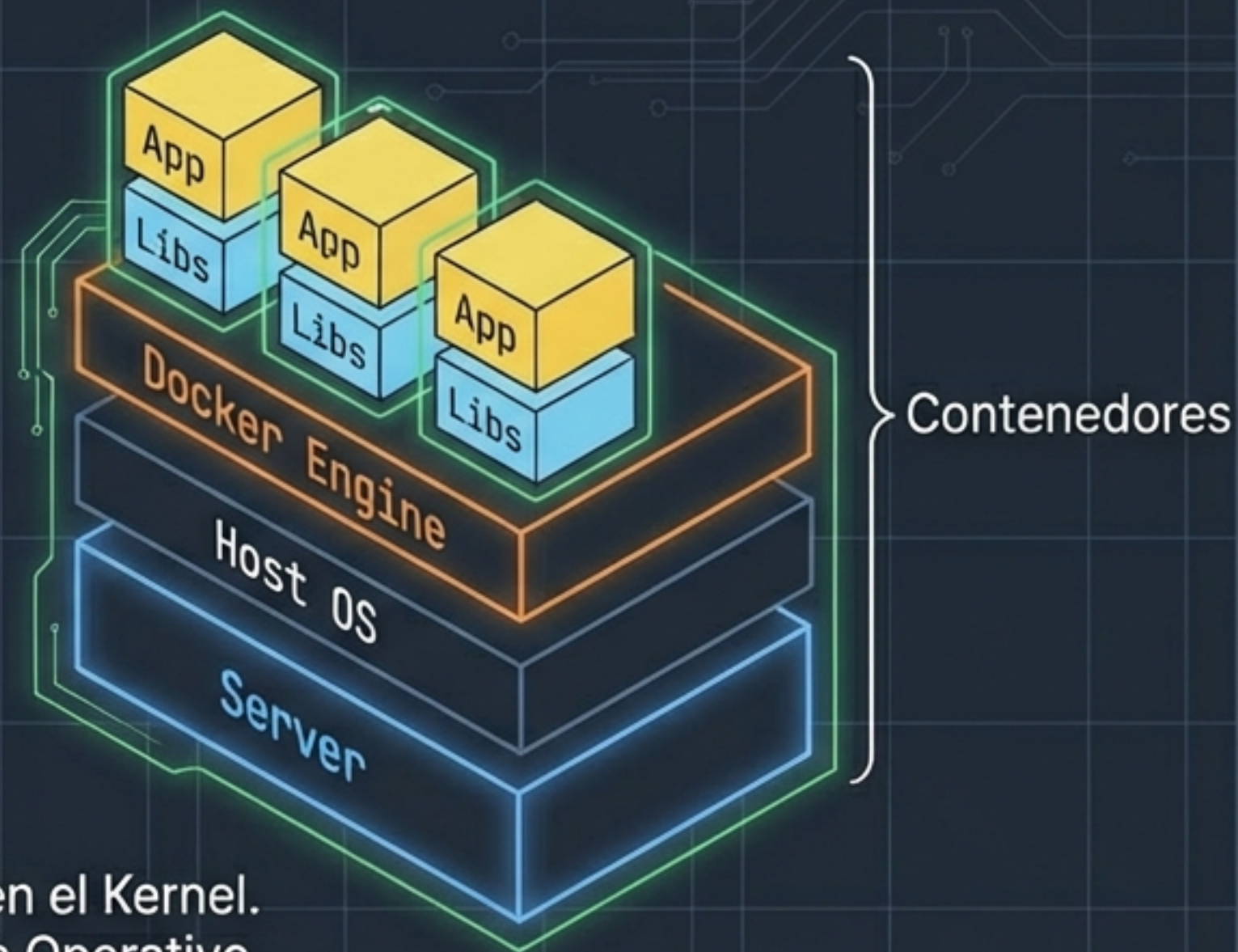
Comandos:
`ps`, `top`, `kill`

La Revolución: Máquinas Virtuales vs. Contenedores

Máquina Virtual (Pesado, GBs)



Contenedor (Ligero, MBs)



Los contenedores comparten el Kernel.
Las VMs duplican el Sistema Operativo.

Arquitectura Docker: Cliente, Demonio y Registro

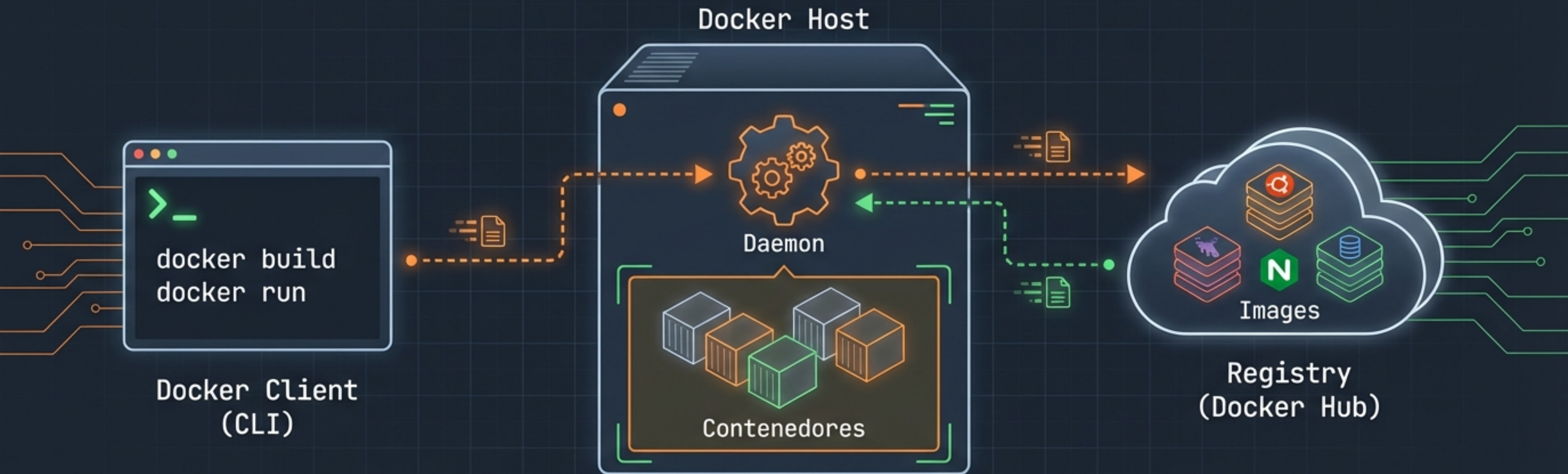
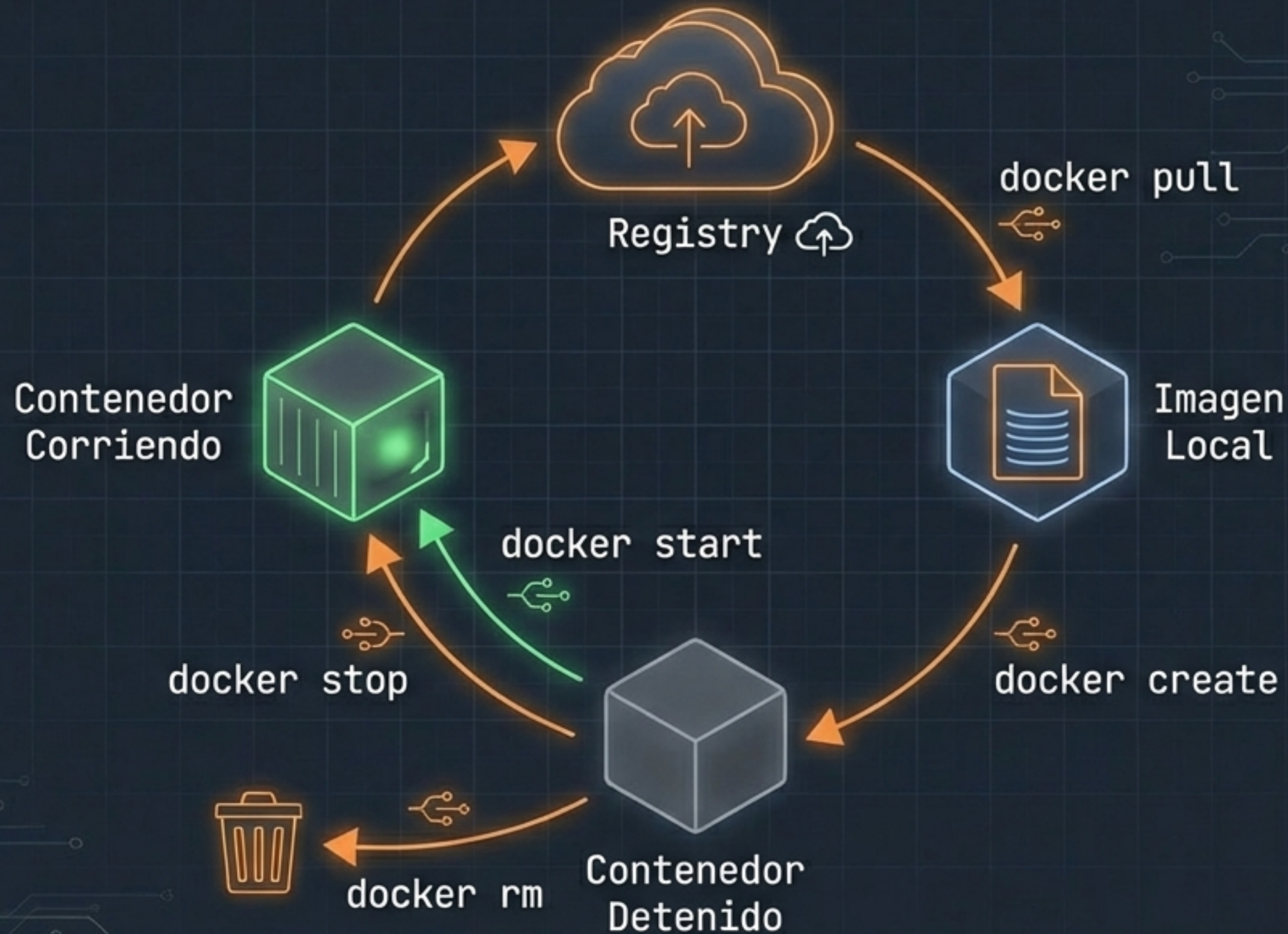


Imagen: El plano (Solo lectura). 

Contenedor: La casa construida (Ejecución). 

Ciclo de Vida del Contenedor



Persistencia y Conectividad

Persistencia de Datos

Mapeo de Puertos

Volumen (-v)

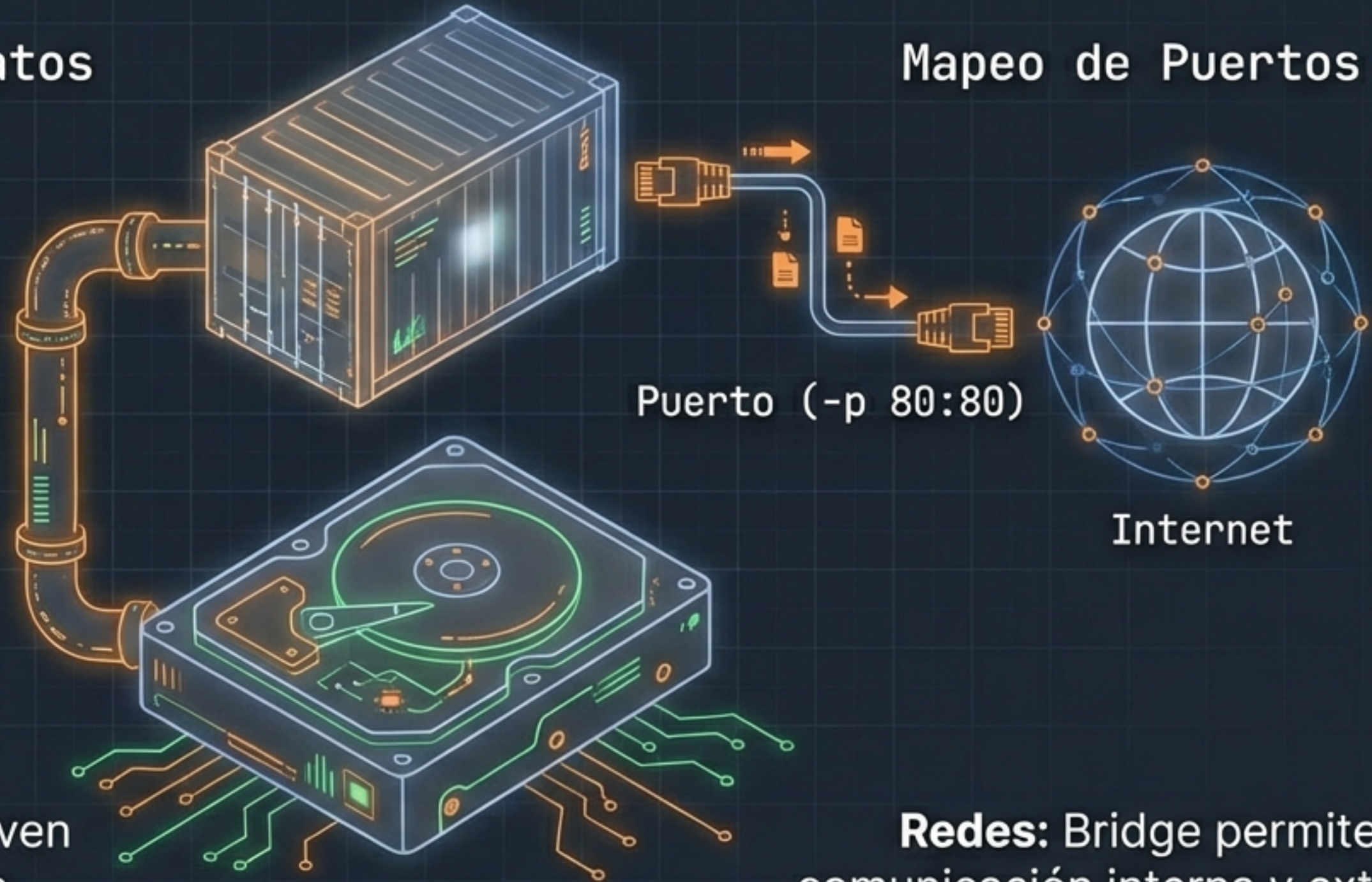
Puerto (-p 80:80)

Internet

Volúmenes: Los datos sobreviven aunque el contenedor muera.

Host Filesystem

Redes: Bridge permite comunicación interna y externa.



Dockerfile: Infraestructura como Código

```
FROM ubuntu:20.04
```

```
COPY . /app
```

```
RUN make /app
```

```
CMD python app.py
```

Dockerfile

FROM: Imagen Base

COPY: Añadir código

RUN: Instalar dependencias

CMD: Comando de inicio

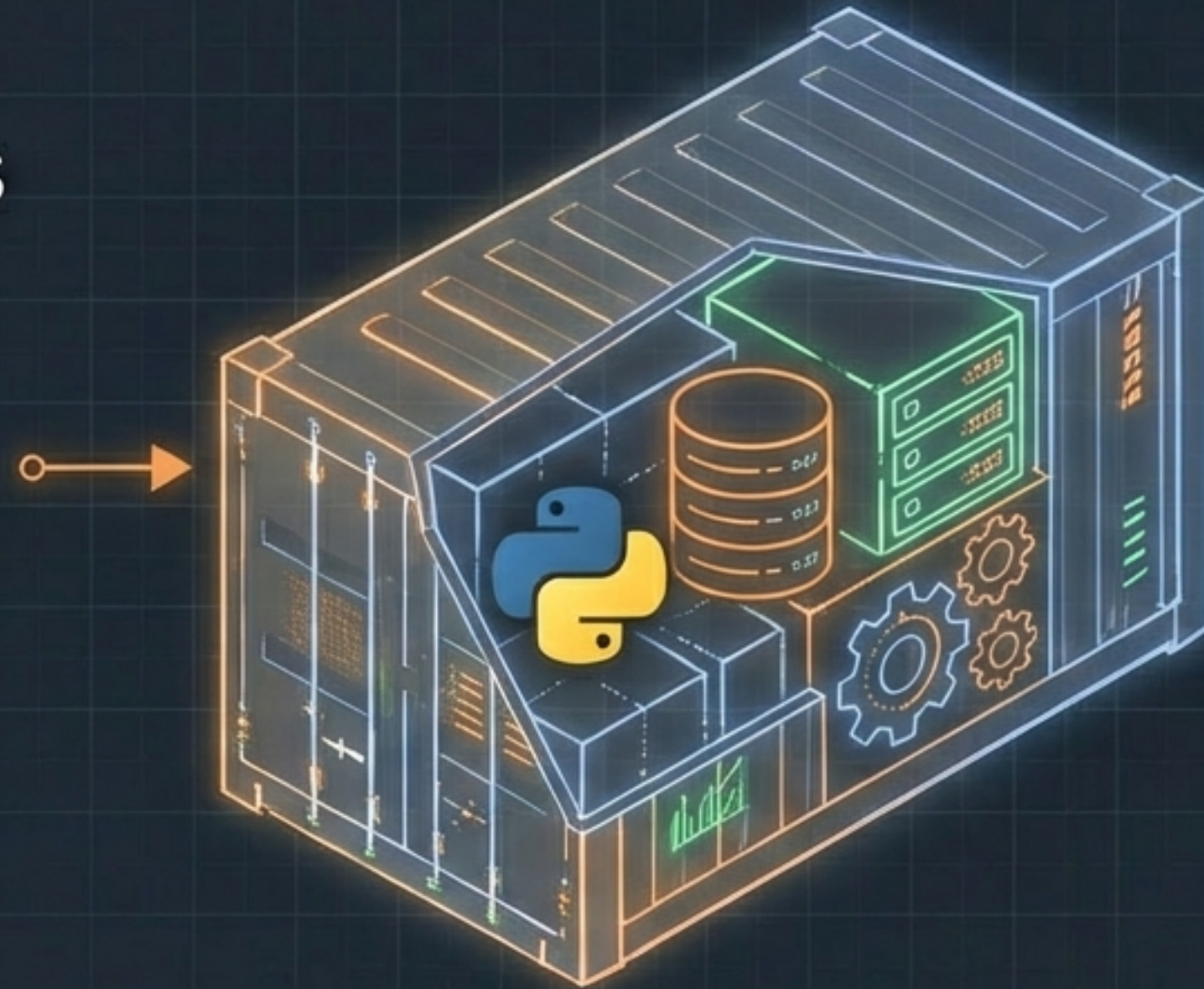
docker build



Imagen Docker

El Desafío del Despliegue Moderno

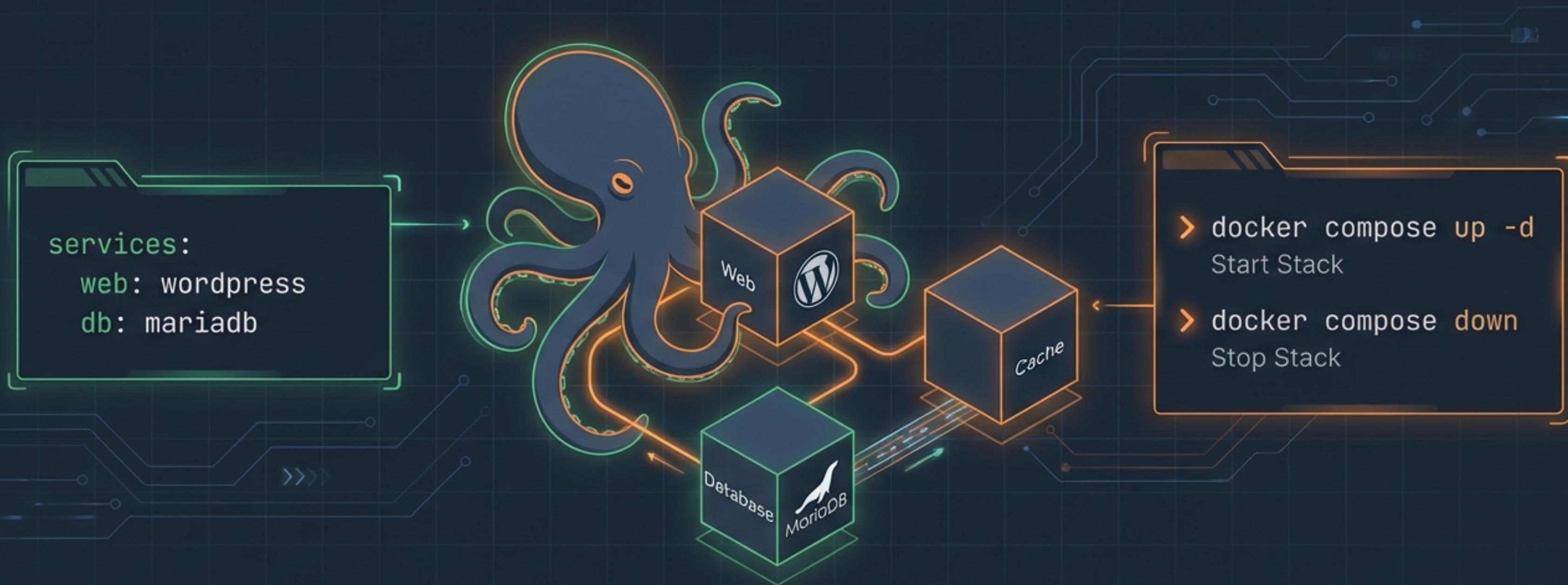
Dependencias
en conflicto
&
“Funciona en
mi máquina”



Entorno
empaquetado
=
Consistencia
en Producción

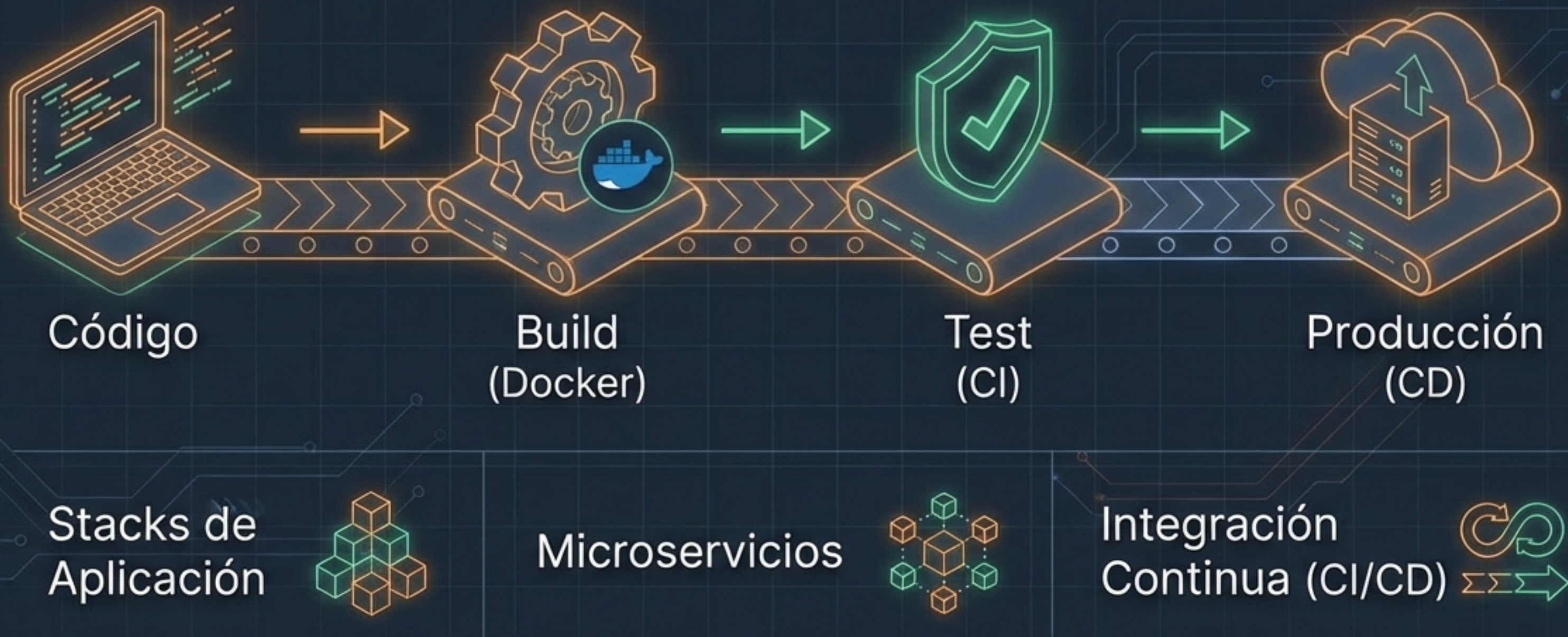


Docker Compose: La Orquestación Simplificada



Un solo archivo para definir toda la arquitectura de la aplicación.

El Futuro: De Compose a Producción



Gestión de infraestructura inmutable, portable y escalable.